

LinkedHashSet

Predicable Iteration Meets Uniqueness

CODEHIVE

CODEHIVE

1. The Best of Both Worlds

The `LinkedHashSet` is a specialized version of `HashSet` that maintains a **doubly-linked list** running through all of its entries. This defines the iteration order, which is the order in which elements were inserted into the set (insertion-order).

The Core Hybrid Concept:

- **From `HashSet`:** It inherits high-speed search and uniqueness.
- **From `LinkedList`:** It inherits predictable ordering.

Unlike a regular `HashSet`, where the order is unpredictable, a `LinkedHashSet` allows you to loop through elements exactly as you added them.

2. Technical Implementation

The following example demonstrates how the `LinkedHashSet` ignores duplicates while preserving the order of the remaining elements.

```
import java.util.LinkedHashSet;

public class Main {
    public static void main(String[] args) {
        LinkedHashSet<String> cars = new LinkedHashSet<>();

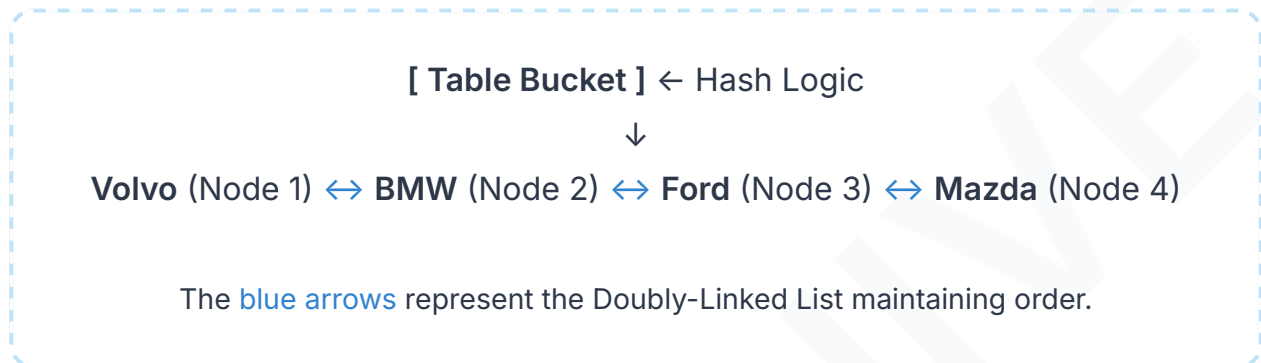
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("BMW"); // DUPLICATE - Ignored
        cars.add("Mazda");

        // Loop: Volvo -> BMW -> Ford -> Mazda
        for(String car : cars) {
            System.out.println(car);
        }

        System.out.println("Total: " + cars.size());
    }
}
```

3. Visualizing the Memory

While a HashSet is just a table of entries, a LinkedHashSet looks more like this in RAM:



If you add "BMW" again, the Hash Logic finds it already exists and does nothing. The links remain unchanged, keeping "BMW" in its original physical position (second in the list).

4. The Complexity Cost

Does the extra "Linkage" make it slow? Not significantly.

- **Time Complexity:** Like HashSet, basic operations (add, contains, remove) remain $O(1)$.
- **Iteration Speed:** Actually faster than HashSet. Iterating over a LinkedHashSet takes time proportional to the number of elements ($O(n)$). HashSet iteration takes time proportional to its capacity (buckets), which is often larger than the number of items.
- **Space Complexity:** Higher. It requires more memory to store the 'before' and 'after' pointers for every node.

5. When To Use LinkedHashSet?

Choose this implementation when the sequence of data is as important as its uniqueness.

Top Use Cases:

- **History Tracking:** Storing unique URL visits while wanting to show them in the order they were visited.
- **Menu Systems:** Building a dynamic list of settings where the order is defined by the user.
- **Data Cleanup:** Removing duplicates from an ArrayList while keeping the original order intact.

6. Comparison: The Set Trio

This table helps you decide between the three main Set implementations in Java.

Property	HashSet	LinkedHashSet	TreeSet
Order	None	Insertion	Sorted
Speed	Fastest	Fast	Moderate
Memory Req.	Low	High	Moderate

7. Modern Syntax & Styles

Just like our previous guides, `LinkedHashSet` supports all modern Java declaration styles.

```
// Professional Interface Style
Set<String> mySet = new LinkedHashSet<>();

// Modern var Keyword (Local scope)
var mySet = new LinkedHashSet<String>();
```

Reminder: If you use `Set<String>` on the left, you can only use methods defined in `Set`. This is good for flexibility, but since `LinkedHashSet` doesn't really have extra "special" methods (unlike `TreeSet`), this is the preferred way to code.

8. Real World Scenario: MRU List

A "Most Recently Used" list ensures that when you duplicate an action, the order doesn't shuffle, but new unique actions are appended predictably.

```
LinkedHashSet<String> recentFiles = new LinkedHashSet<>();

void openFile(String path) {
    recentFiles.add(path); // Moves nowhere if exists, adds to end if new
}
```

Note: Regular `LinkedHashSet` adds new items to the end. If you want a true move-to-front behavior, you would need to `remove()` and then `add()` the element again.

9. Summary & Final Thoughts

Success Checklist:

- Use **LinkedHashSet** when you need a fast set that remembers when items were added.
- Be mindful of memory if you are storing millions of elements.
- It is the perfect "Cleanup Tool" to deduplicate a list while keeping the user's order.

© 2026 CodeHive Academy

Java Collections Documentation Series - Conclusion